

On the use of test smells for prediction of flaky tests

Bruno Henrique Pachulski Camara
Department of Computer Science,
Federal University of Paraná
Curitiba, PR, Brazil
bhpachulski@utfpr.br

Andre T. Endo
Department of Computing,
Federal University of Technology - Paraná
Cornélio Procópio, PR, Brazil
andreendo@utfpr.edu.br

Marco Aurélio Graciotto Silva
Department of Computing,
Federal University of Technology - Paraná
Campo Mourão, PR, Brazil
magsilva@utfpr.edu.br

Silvia Regina Vergilio
Department of Computer Science,
Federal University of Paraná
Curitiba, PR, Brazil
silvia@inf.ufpr.br

ABSTRACT

Regression testing is an important phase to deliver software with quality. However, flaky tests hamper the evaluation of test results and can increase costs. This is because a flaky test may pass or fail non-deterministically and to identify properly the flakiness of a test requires rerunning the test suite multiple times. To cope with this challenge, approaches have been proposed based on prediction models and machine learning. Existing approaches based on the use of the test case vocabulary may be context-sensitive and prone to overfitting, presenting low performance when executed in a cross-project scenario. To overcome these limitations, we investigate the use of test smells as predictors of flaky tests. We conducted an empirical study to understand if test smells have good performance as a classifier to predict the flakiness in the cross-project context, and analysed the information gain of each test smell. We also compared the test smell-based approach with the vocabulary-based one. As a result, we obtained a classifier that had a reasonable performance (Random Forest, 0.83%) to predict the flakiness in the testing phase. This classifier presented better performance than vocabulary-based model for cross-project prediction. The *Assertion Roulette* and *Sleepy Test* test smell types are the ones associated with the best information gain values.

KEYWORDS

test flakiness, regression testing, test smells, machine learning

ACM Reference Format:

Bruno Henrique Pachulski Camara, Marco Aurélio Graciotto Silva, Andre T. Endo, and Silvia Regina Vergilio. 2021. On the use of test smells for prediction of flaky tests. In *Brazilian Symposium on Systematic and Automated Software Testing (SAST'21)*, September 27-October 1, 2021, Joinville, Brazil. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3482909.3482916>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SAST'21, September 27-October 1, 2021, Joinville, Brazil

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8503-9/21/09...\$15.00

<https://doi.org/10.1145/3482909.3482916>

1 INTRODUCTION

Regression testing is an important phase to deliver software continuously with quality and minimizing failures after changes in the production code. In this phase, developers evaluate the test results to decide whether the program has a bug as a consequence of recent changes. However, the existence of flaky tests makes this evaluation untrustworthy. This happens because some tests have an intermittent behavior, that is, they pass and fail when executed in the same codebase [11, 16]. This non-deterministic behavior, alternating between passing and failing without any code changes, frustrates developers. Flaky tests are challenging to debug, and a single failing test can halt release cycles [2].

Studies from literature show that flaky tests are common and appear in most large-scale projects [6]. In such cases, developers may spend important resources in analyzing failures that are due to flaky tests and not to actual problems in the code. Practitioners get now used to rerun each newly observed failure several times, to ascertain that it is a genuine regression failure and not an intermittent one [25]. This negatively impacts the productivity and contributes to increase costs.

A recent review on flaky tests points a growing interest in this subject [28]. The academic and industrial software development communities have worked towards the definition of flaky tests [14–16, 19, 21], reporting their occurrence [6, 13], building datasets [2, 3, 15, 25], and finding automatically ways to identify them [1, 2, 4, 10, 10, 12, 20, 25]. The identification of flaky tests is usually addressed by two kinds of approaches. Dynamic approaches usually re-execute the test cases a fixed number of times [15, 26]. This is expensive and error-prone. Moreover, it is not easy to determine how many executions is enough. This can be challenging and an inadequate choice can lead to false negatives. Static approaches in turn do not require code re-execution [6, 12, 16, 17, 25, 26].

Most of the works apply Machine Learning (ML) methods to build models to predict flakiness likelihood [2, 4, 12, 17, 25]. These works differ on the adopted ML method and features used as predictors. Some of them use as predictors features obtained through the execution of code, such as coverage and runtime [2, 12].

Models built using only static features present many advantages and are less costly [25]. Considering this advantage, Pinto et al. [25] build the set of predictors considering that there are some patterns within the test code that may be employed to automatically identify

flaky tests. Then, tokens are identified and processed by using Natural Language Processing (NLP) techniques to synthesize a vocabulary of flaky tests. This vocabulary is used as predictors in combination with some common static test case features regarding number of lines of code and occurrence of certain Java keywords. This work applied five ML algorithms and Random Forest and Support Vector Machine (SVM) reached the best performance (F-measure of 0.95).

The works of Ahmad et al. [1] and Haben et al. [10] attempted to reproduce the findings of Pinto et al. and evaluated the effectiveness of vocabulary-based features in Python projects. Haben et al. [10] also used an information retrieval technique to statically estimate testing coverage, and investigated whether it could improve the performance of a vocabulary-based model, but without success. Camara et al. [4] replicated the work of Pinto et al. and also investigated the generalization of the original results in real scenarios. By using different datasets, the authors evaluated the performance of the vocabulary-based model for a cross-project context, considering intra- and inter-project test flakiness prediction. The authors concluded that the vocabulary-based approach is context-sensitive and prone to overfitting, presenting low performance when executed in a cross-project scenario.

Considering these negative results, this work investigates the use of an alternative approach for flaky test prediction, based on test smells. Similarly to the concept of bad code smells [8], test smells are associated to potential design problems in the test code, that is, problems related to the way test cases are organized, implemented and interact with each other [5]. Test smells may also impact the software quality and has been recently associated to test flakiness. For instance, Alshammari et al. [2] investigated the use of test smells as predictors of flaky tests but in combination with other features. The set of features used included static metrics such as number of lines and assertions but also dynamics metrics related to line coverage, what can increase costs.

Unlike the work of Alshammari et al. we adopted a set of predictors only composed by metrics collected statically. In addition to the size of test case and number of smells in the test code, we also adopted binary features related to the presence or not of 19 test smells. Moreover, we investigated the use of the obtained models for cross-project prediction in comparison with the vocabulary-based approach. To this end, we employed the same dataset and algorithms used in our previous work [4]. As a result, we obtained a classifier that has a reasonable performance (Random Forest, 0.83%). This classifier presented better performance than vocabulary-based model for cross-project prediction. The *Assertion Roulette* and *Sleepy Test* smells are the ones associated with the best information gain values. In this way, the main contributions of this paper are:

- An empirical study of the adoption of test smell-based models for the prediction of test flakiness. Such an approach presents some advantages. The use of smells can be collected statically and automatically. In our work we use the tsDetect tool [24];
- Results about the importance of the different type of smells and their relation with flaky tests. This can guide testers to better design test code;

- Discussion on some implications and limitations about the generalization of the results and obtained models for different projects in a cross-validation scenario, that can guide future research in the area; and
- A repository containing the procedures, datasets, and scripts generated from this study at <https://github.com/bhpachulski/SAST21-Paper>.

The paper is organized as follows. Section 2 provides background about test smells and flaky tests. Section 3 contains an example showing how smells and flakiness may relate. This example serves as motivation to our work. Section 4 reviews related work. Section 5 describes the methodology adopted in our study. Section 6 presents and analyses the obtained results. Section 7 discusses the main threats of our study. Section 8 presents our final remarks and concludes the paper.

2 BACKGROUND

In this section we provide an overview of the main topics related to this work: flaky tests and test smells.

2.1 Flaky test

In regression testing, test suites are executed to validate if code changes, like new features added or bug fixing, do not negatively impact the software. However, not all test failures during the regression testing uncover new faults on production code [11]. Some tests have an intermittent behavior, that is, in a given moment they execute with success, but in others they fail for the same code version. A test with this characteristic, which passes/fails non-deterministically, is known as flaky [16].

Figure 1 illustrates a flaky test reported by Lam et al. [15]. A time limit (timeout=2000) to execute the method testIssue is defined in Line 1. In this way, if the method takes more than two seconds (i.e., 2000 ms), the test case fails. In some cases, the test case is executed with success, but eventually, if the resource that is required is not ready, the test case will fail. In other words, the test case behavior can be associated with the environment, hardware, and some other reasons [15]. The existence of flaky test is inconvenient for software development and regression testing activities. Many times, to reproduce the failure that is non-deterministic, and to detect flakiness it is necessary to re-execute the code and this can be costly for the software development process.

```

1 @Test(timeout=2000)
2 public void testIssue() throws Exception {
3     final int port = SocketUtil.getAvailablePort();
4     WebSocketServer server = new WebSocketServer(
5         new InetSocketAddress(port)) { ...}
6     ...
7 }

```

Figure 1: Flaky test example [15].

Luo et al. [16] identified a list of the most prominent categories of flaky tests for developers and researchers to focus on. As a result of this study a classification of the main causes of flakiness was introduced. Figure 1 describes the causes found. The most prevalent root causes are *Async Wait* with 45% of the cases, *Concurrency* with 20%, and *Test Order Dependency* with 12%. These three causes represent 77% of the cases [16].

Table 1: Classification for the causes of flakiness, proposed by Luo et al. [16].

Cause	Description
<i>Async Wait</i>	When the test makes an asynchronous call and does not wait for the result to be available before using it.
<i>Concurrency</i>	When the test starts several threads that interact non-deterministically, causing a undesirable behavior.
<i>Test Order Dependency</i>	When the test outcome depends on the order in which the test cases are run.
<i>Resource Leak</i>	When the test does not acquire or release resources, e.g., memory allocations or database connections.
<i>Network</i>	When the test execution depends on the network and can be flaky because the network is hard to control and unpredictable.
<i>Time</i>	When the test assertion is based on a time or due to the precision by which time is reported as it can vary inter-platforms.
<i>IO operations</i>	When the test needs to access an IO resource that may also cause flakiness.
<i>Randomness</i>	When the test assertion is based on a random number or information, this may be a potential flaky.
<i>Floating Point Operations</i>	When dealing with floating point operations is known to lead to tricky non-deterministic cases.
<i>Unordered Collections</i>	When iterating over unordered collections the code should not assume that the elements are returned in a particular order.

2.2 Test smells

Test code, just like any production code, is subject to be poorly written, without taking good programming practices into account, fact that introduces the so-called anti-patterns or smells [8]. Test smells are a deviation of how the tests should be written, organized and how tests should interact with others. That deviation can indicate test design problems, and can hurt the test performance [5, 23]. This section gives an overview of bad code smells that are specific for test code.

Initially, Deursen et al. [5] proposed a set of test smells composed of *Assertion Roulette*, *Eager Test*, *General Fixture*, *Lazy Test*, *Mystery Guest*, *Resource Optimism*, and *Sensitive Equality*. Then, Peruma et al. [23] extended the types of test smells with others inspired by bad test programming practices mentioned in unit testing based literature. In a later work, Peruma et al. [24] introduced tsDetect, an open source test smells detection tool. Table 2 presents definitions and detection rules of test smells used by tsDetect. This state-of-the-art tool detects a comprehensive set of test smells and was adopted in our study.

3 MOTIVATING EXAMPLE

Considering the causes of flakiness described by Luo et al. [16] and test smells, we can observe it is possible to make relations between some of them. For example, instructions that inject delays may be related to the flakiness root causes *Async Wait* or *Concurrency*; it is possible to wait for an Async operation to complete or provide delays to synchronize two or more threads. The use of delays is related to the *Sleepy Test* smell.

To illustrate this, Figure 2 shows a method from the test class TESTMEMORYLOCKS, from the open-source project Oozie¹, a workflow scheduler system to manage Apache Hadoop² jobs. This example is a flaky test, extracted from the dataset provided by Pinto et al. [25]. Using the tsDetect tool [24] in the test method TESTREADWRITELOCK, two test smells are detected: *Sensitive Equality*, and *Sleepy Test*.

The test starts instantiating a STRINGBUFFER object to be used by the code under test and on the assertion (Line 3). Then, two objects from classes READLOCKER and WRITELOCKER are instantiated (Lines 4 and 5); these objects consist of the main production code under test. A thread with READLOCKER is started in Line 7 and, after a sleep (Line 8). The same occurs to the WRITELOCKER object (Lines 9 and 10). Similarly, both locker objects are finished (Lines 11 to 14). Finally, the assertion checks whether the simulated behavior with read and write operations to variable sb is accurate or not (Line 15).

```

1 @Test
2 public void testReadWriteLock() throws Exception {
3     StringBuffer sb = new StringBuffer("");
4     Locker l1 = new ReadLocker("a", 1, -1, sb);
5     Locker l2 = new WriteLocker("a", 2, -1, sb);
6
7     new Thread(l1).start();
8     Thread.sleep(500);
9     new Thread(l2).start();
10    Thread.sleep(500);
11    l1.finish();
12    Thread.sleep(500);
13    l2.finish();
14    Thread.sleep(500);
15    assertEquals("a:1-L a:1-U a:2-L a:2-U", sb.
        toString().trim());
16 }

```

Figure 2: A flaky test from the Oozie project [25].

The root cause for flakiness is probably related to *Concurrency* due to the use of delays associated with the threads. This test code contains the smell *Sleepy Test*, since it uses the SLEEP method on a THREAD. Also, the *Sensitive Equality* smell is identified by the use of a toString method in the assertion. As defined by Luo et al. [16], the root cause of flakiness *Concurrency* happens when the test starts several threads that interact non-deterministically, causing the intermittent behavior.

Just like this example, other four test cases are identified as flaky in the same class; their structure are pretty similar. This example indicates that the root cause of a flaky test may be associated to the presence of one or more test smells. Test cases that have this

¹<https://oozie.apache.org/>

²<https://hadoop.apache.org/>

Table 2: Test smells and detection rules of tsDetect [24].

Test Smell	Detection Rule
<i>Assertion Roulette</i>	It occurs when a method has more than an assertion, so if one fails it is difficult to define which one.
<i>Conditional Test Logic</i>	A test method that contains control flow statements (i.e IF, SWITCH, conditional expression, FOR, FOREACH and WHILE statements).
<i>Constructor Initialization</i>	A test class that contains a constructor declaration.
<i>Default Test</i>	A test class is named either “ExampleUnitTest” or “ExampleInstrumentedTest”.
<i>Duplicate Assert</i>	A test method that contains more than one assertion statement with the same parameters.
<i>Eager Test</i>	A test method that contains multiple calls to multiple production methods.
<i>Empty Test</i>	A test method that does not contain a single executable statement.
<i>General Fixture</i>	Not all fields instantiated within the <code>setUp</code> method of a test class are utilized by all test methods in the same test class.
<i>Ignored Test</i>	A test method or class that contains the <code>@Ignore</code> annotation.
<i>Lazy Test</i>	Multiple test methods calling the same production method.
<i>Magic Number Test</i>	An assertion method that contains a numeric literal as an argument.
<i>Mystery Guest</i>	A test method containing object instances of files and databases classes.
<i>Redundant Print</i>	A test method that invokes either the <code>print</code> or <code>println</code> or <code>printf</code> or <code>write</code> method of the <code>System</code> class.
<i>Redundant Assertion</i>	A test method that contains an assertion statement in which the expected and actual parameters are the same.
<i>Resource Optimism</i>	A test method that utilizes an instance of a <code>File</code> class without calling the <code>exists()</code> , <code>isFile()</code> or <code>notExists()</code> methods of the object.
<i>Sensitive Equality</i>	A test method that invokes the <code>toString()</code> method of an object.
<i>Sleepy Test</i>	A test method that invokes the <code>Thread.sleep()</code> method.
<i>Unknown Test</i>	A test method that does not contain a single assertion statement and <code>@Test(expected)</code> annotation parameter.
<i>Verbose Test</i>	A test that is too long and hard to understand [18, 27].

pattern are identified just by re-executing the code. This process is expensive and time-consuming, as demonstrated by Pinto et al. [25] that found around 70% of flaky test cases passed in more than 90% of the executions. This fact serves as a motivation to use test smells as predictors of test flakiness.

4 RELATED WORK

The presence of flaky tests may imply extra effort during the software engineering process. For instance, the developer can spend a lot of time debugging a failing test case that happens to be flaky. On the other side, ignoring a failing test by misclassifying it as flaky would cause the shipping of faulty software to the users. Flaky tests may occur due to software changes, though other causes have been identified in the literature [6, 16]; see Table 1.

In general, test flakiness detection has brought a lot of attention from industry and academia [7, 9, 19, 21, 28]. One direction is to adopt dynamic approaches whose core involves rerunning test cases for a fixed number of times [15, 26]. A clear disadvantage is the cost of execution; for large test suites, this strategy may not scale. A different direction is to rely on statically-extracted information.

Works in the literature that are most related to ours apply Machine Learning (ML) methods to detect flaky tests [2, 4, 12, 17, 25]. Memon et al. [17] introduce an approach to minimize the workload of the test automation platform in Google. To do so, the approach avoids the execution of test cases with low failure probability, and present insights to developers in order to prevent bugs. For ML part, the following features are used: CI tool, transitions PASSED-to-FAILED, fixes FAILED-to-PASSED, and developers’ activity. The

results show that the number of test runs can be reduced while maintaining similar bug detection capabilities.

King et al. [12] apply Bayesian networks to predict flaky tests. The used features are a mix of static and dynamic metrics: (i) complexity: assertion count, test class/method size, depth of inheritance tree, (ii) implementation coupling: coupling between objects and selector stability index, (iii) non-determinism: cyclomatic complexity and explicit wait count, (iv) performance: average execution time, and (v) general stability metrics: failure rate and flip rate. The authors evaluated the proposed approach with a case study with five teams developing a proprietary Web application. During the study, the approach supported the reduction of flaky tests; for some cases, the reduction was up to 60%. Overall, the accuracy of the prediction was 65.7%.

Pinto et al. [25] propose that there exists a vocabulary of patterns (words) in the test code that can be extracted using NLP to predict whether a test is flaky or not. To evaluate the approach, the authors constructed a dataset of Java projects, with test cases labelled as flaky and non-flaky, to train and test ML algorithms. Overall, all classifiers had good performance: Random Forest with the best precision (0.99) and F₁-Score, and SVM with the best recall (0.92). The paper also shows the top-20 features with the highest information gain. Other studies also investigated how the vocabulary-based approach performs in Python projects [1, 10].

In a previous paper [4], we conducted a replication of the Pinto et al. [25]’s study. The main extension was to assess the trained classifiers to predict flaky tests using a different test dataset (not used for training) in two contexts: For the intra-project context, the dataset contained tests from the same projects used during

the training, while tests from different projects were used for the inter-project context. Among the trained classifiers, the best one was LDA with recall of 0.75 for intra-project, and 0.45 for inter-project. The authors conclude that the vocabulary-based approach is context sensitive and prone to overfitting.

In the same line, Alshammari et al. [2] introduced an approach called FlakeFlagger that employs static and dynamic features like test smells, test coverage, source code management system, and source code, to predict test flakiness. Using a newly-defined dataset with 24 open source Java projects, the authors compared FlakeFlagger with the vocabulary-based approach [25], and a combination of both. The obtained recall was similar for the 3 approaches compared (74%, 72%, and 74%), but the precision has a 49% difference between the FlakeFlagger and vocabulary-based, and the combined approach has a sensitive improvement of 6%. So, the precision of the vocabulary-based approach was lower than FlakeFlagger. As for test smells, FlakeFlagger implements its own detector using an expanded definition of existing test smells. By analyzing the information gain, the results showed very low correlation with test flakiness.

We can see that the use of test smells is few explored in the literature. We find only one work that consider test smells [2], but in combination with other predictors, such as coverage that need at least one execution of the code. Differently from related work, we adopt only static metrics and perform a validation of the obtained models in cross-project scenario.

5 METHODOLOGY

The main goal of this study is to investigate the use of test smells as predictors of flakiness. The main advantage of this approach is that the smells can be collected statically. In addition to this, we investigate the use of the smell-based models as an alternative for a cross-project scenario.

5.1 Research questions

According to our goals we defined three Research Questions (RQs).

- **RQ₁.** How accurately can we predict test flakiness based on test smells in the test cases? The goal of this question is to evaluate the performance of classifiers to predict test flakiness based on the presence/absence of test smells, without re-execution of the test suites.
- **RQ₂.** Which test smells are the most strongly associated with test flakiness prediction? The goal is to identify the test smells which are more related to flakiness in order to help development, code review, and debugging tasks.
- **RQ₃.** How does the test smell-based approach compare with the existing vocabulary-based approach? The goal was to compare the obtained results with the vocabulary-based approach [25], which can be considered as state-of-the-art approach that also does not require test re-execution.

RQ₁ and RQ₃ are answered considering two perspectives. In the first perspective a prediction model is built and evaluated according to some performance measures and compared with the vocabulary-based approach. For this end, the dataset made available by Pinto et al. [25] is used. In the second perspective, the evaluation considers the cross-project scenario. To this end, the dataset as extended by

our previous work Camara et al. [4] is used. In this cross-project perspective, we evaluate the performance of the built models with different datasets: i) within a different set of test cases from the same software projects (intra-projects); and ii) within other different projects (inter-projects).

The next sections present details about the methodology adopted to answer the RQs considering both perspectives: datasets, classifiers used to build the models, and evaluated measures.

5.2 Datasets

As mentioned before, for building the models, we use the dataset of Pinto et al.'s work [25], built based on 24 DeFlaker projects [3]. The raw dataset has 49,919 test cases: 44,428 non-flaky, 5,069 flaky. To evaluate the models in the cross-project perspective we used the data from a previous work [4], built based on idFlakies projects of Lam et al. [15]. This dataset contains only flaky tests, in a total of 422, from 72 different projects.

Both datasets were submitted to the tsDetect tool [24]. In a first step this tool requires, for each test case, the identification of the corresponding production code, to then detect the smells the test contains. But for some test cases the code could not be identified, and as a consequence, the smell detection step could not be performed. Because of this, these test cases were removed from both datasets. At the end the dataset from Pinto et al.'s work [25] dropped to 14,390 samples (11,319 non-flaky, 2,914 flaky), to deal with the imbalanced dataset, a number of non-flaky tests was selected randomly, equal to the number of flaky tests. In the dataset based on idFlakies projects Lam et al. [15], the resultant sample dropped to 155.

As result we obtained a list with 19 test smells we used as features for the model; the test smells are described in Table 2. The information about the smells is then augmented with two numerical features, acting as proxies of code complexity: LOC: number of lines of code of the test case; and smells count: total number of test smells present in the test case. The vocabulary-based approach was applied according to related work [4].

Following the methodology adopted in [4], we generated two datasets: one for training and testing the models, and other to cross-project validation. The training and testing dataset is balanced and contains 2,777 samples, 1,377 flaky and 1,400 non-flaky, from 22 projects. This sample size was defined to be compatible with Pinto et al.'s work [25] so we can use that work as benchmark. The cross-project validation dataset was divided to attend the intra- and inter-project context. To obtain the intra-project dataset we filtered a set of flaky tests from 24 projects of the training dataset. At the end this set was composed by 35 samples. We obtained the dataset for the inter-project context by filtering out tests from projects present in the training dataset, this set are composed by 120 samples. In both only flaky samples are present.

5.3 Used Classifiers

We use eight classifiers available in the framework Scikit-learn [22]: Random Forest, Decision Tree (DT), Naive Bayes, Support Vector Machine (SVM), Logistic Regression (LR), Linear Discriminant Analysis (LDA), K-Nearest Neighbour (KNN), and Perceptron. For

comparison reasons, the choice of these classifiers, as well as the used parameters are based on related work [4, 25].

5.4 Evaluated metrics

To evaluate the performance of the classifiers, the dataset was split into 80% for training and 20% for testing. We used the following standard metrics:

- precision: the number of correctly classified flaky tests divided by the total number of tests that are classified as flaky;
- recall: the number of correctly classified flaky tests divided by the total number of actual flaky tests in the test set;
- F_1 -Score: the harmonic mean of precision and recall;
- MCC (Matthews correlation coefficient): measures the correlation between predicted classes (i.e., flaky vs. non-flaky) and ground truth. Values of MCC vary in the interval of $[-1, 1]$, with 1 representing a perfect prediction;
- AUC (area under the ROC curve): measures the area under the curve which visualizes the trade-off between true-positive and false-positive rates.

Concerning the cross-project perspective (namely, intra- and inter-project validation), the idFlakies dataset was adopted, and to evaluate the results only recall were used because, as mentioned, this dataset does not contain examples of non-flaky tests. To evaluate the relevance of the features (RQ2) we utilized the information gain (known as entropy), calculated for each output variable. This value ranges from 0 (no gain) to 1 (maximum of information gain). It was calculated by using the method `mutual_info_classif` of Scikit-learn with default settings.

6 ANALYSIS OF RESULTS

This section analyses the obtained results to answer the research questions.

6.1 RQ₁ – How accurately can we predict test flakiness based on test smells in the test cases?

Following the experimental design described, we first built the prediction model by training and testing the classifiers. The results of the eight classifiers are presented in Table 3. All classifiers archived reasonably performance. Except by Naive-Bayes, they reached values greater than 70% of precision, recall, F_1 -Score, and AUC. Again, except Naive-Bayes, all the classifiers obtained MCC values greater than 0.5 (which are close to 1 that represents a perfect classification).

The obtained results show that test smell-based models have reasonable performance to predict test flakiness, with precision values varying from 74 to 83%. The best classifier was obtained with Random Forest and Decision Tree, which reached similar values for all measures, with precision value of 83%. Naive-Bayes presented the worst performance.

To validate the performance of the model in the cross-project context we tested the trained classifiers utilizing the flaky tests identified in the idFlakies dataset [15] (cross-validation dataset). Table 4 shows the results of intra- and inter-project contexts: it presents the classifiers’ performance considering the recall, true positives (TP), and false negatives (FN).

Table 3: Test smells-based classifiers’ performance.

Algorithm	Precision	Recall	F_1	MCC	AUC
Random Forest	0.83	0.83	0.83	0.65	0.90
Decision Tree	0.83	0.83	0.83	0.66	0.86
KNN	0.81	0.81	0.81	0.62	0.81
LR	0.79	0.79	0.79	0.59	0.87
LDA	0.78	0.78	0.78	0.56	0.86
Perceptron	0.78	0.78	0.78	0.55	0.86
SVM	0.75	0.75	0.75	0.50	0.83
Naive Bayes	0.74	0.65	0.61	0.37	0.78

Table 4: Cross-project test smells-based classification.

Algorithm	Intra-Project			Inter-Project		
	Recall	TP	FN	Recall	TP	FN
Random Forest	0.69	24	11	0.54	65	55
Decision Tree	0.66	23	12	0.54	65	55
KNN	0.51	18	17	0.51	61	59
LR	0.74	26	9	0.48	57	63
LDA	0.66	23	12	0.48	57	63
Perceptron	0.71	25	10	0.48	57	63
SVM	0.66	23	12	0.55	66	54
Naive Bayes	0.57	20	15	0.14	17	103

Considering the intra-project context, the performance of all classifiers dropped to recall values varying from 51% to 74%, being the best value reached by LR that classified correctly 26 of 9 flaky tests. The performance of the classifiers in the inter-project context dropped more significantly. But, excluding Naive-Bayes that reached the value of 14%, there is not great difference between the classifiers, with recall values varying from 48% to 55%. SVM reached the best performance.

Answer to RQ₁: The obtained results show that test smells can be used as predictors of flakiness. Nevertheless, the performance drops considerably in the inter-project context.

Implications: The results show that the smell-based models present performance comparable and in some cases better than some values reported in the literature (precision around 70%) [2, 4, 12]. This lead to the conclusion that smells are good predictors of flakiness, but future work should explore the use of smells with other features, obtained statically. This can contribute for improving performance in the cross-project validation and for obtaining more generalizable models.

6.2 RQ₂ – Which test smells are the most strongly associated with test flakiness prediction?

The features most associated with test flakiness were determined by calculating the information gain based on the entropy of the features. Table 5 shows all features adopted in the proposed model, ordered by their relevance (i.e., information gain); the data is based

on the training dataset. Column ‘Total’ shows how many tests are affected by the feature, while columns ‘Flaky tests’ and ‘Non-Flaky tests’ bring the number of affected tests that are flaky or not, respectively; the percentages are presented in the next columns.

Considering the contribution of the test smells for the model, we observe four smells with at least 90% of affected tests being flaky. They are: `Verbose Test` (100%), `Redundant Print` (94.83%), `Sleepy Test` (93.75%), and `Constructor Initialization` (92.65%). Of these, `Sleepy Test`, and `Constructor Initialization`. These smells can be associated with types of flaky tests.

The `Sleepy Test` smell is related to the use of delays (`Thread.sleep()` or similar statements) to wait for other components to be ready to be executed. That behavior is described by Luo et al. [16] as the flaky test type `Async Wait`. In Table 8 `Sleepy Test` is found in 105 (94%) flaky tests and in just 7 non-flaky tests. `Constructor Initialization` occurs when a test class has a constructor, which also can be related to the flakiness type `Test Order Dependency`, when the test result depends on the order in which the tests are run. This test smell type are found in 93% of flaky tests and 6% in non-flaky tests. The `Assertion Roulette` smell occurs when the method has more than an assertion; this is the test smell most found in our study, in 69.69% of the cases they are associated with flaky tests and is the second feature with biggest information gain. The average of lines of code of methods that are identified with `Assertion Roulette` is over 28, containing 3.4 smells.

Table 6 shows the smell count distribution over the training dataset, in which we can see the greater the number of smells the greater the flakiness percentage. In our dataset we have 122 test cases without test smells, 17% are identified as flaky, when considering test cases with one smell, the percentage grows to 39%, becoming at 71% with six smells. However, if a group with 1 to 4 test smells is considered we have a set of 89% of the data, 50% of flaky, and 50% of non-flaky tests, that is, future work is necessary to better investigate if the feature smell count is a good feature to identify flaky test.

Answer to RQ₂: `Sleepy Test`, and `Constructor Initialization` can be associated with flaky test types. This is demonstrated in our dataset by the distribution of numbers in flaky tests and by the information gain.

Implications: The information gain presents the ordered set of features that are more discriminant; it is possible to see some features that make sense but more studies are necessary to analyze if in fact all of them have a positive impact in the prediction.

6.3 RQ₃ – How does test the smell-based approach compare with the existing vocabulary-based approach?

Using the vocabulary-based approach we trained the eight classifiers with our training and testing dataset (the same used in RQ₁) to answer RQ₃. Table 7 shows the results of the trained classifiers. Despite we are using distinct datasets, as explained in Section 5.2 the results are similar to those presented in previous studies [4, 25].

After this, we apply the vocabulary-based approach using our cross-validation dataset. The results of the classifiers obtained for

the cross-project contexts are presented in Table 8. The obtained performance of intra-project classifications is worst than the performance reported previously [4] where the best classifier was LDA with 75% of recall classifying 20 of out 60 samples correctly. Here, the best classifier was KNN with 57% of recall, classifying correctly 20 of out 45 samples. The inter-project performance was slightly better than in [4], obtaining 56% of recall against 48% (122 of 134).

Comparing with the smell-based approach, the vocabulary-based approach presents better performance: the best F₁ score of vocabulary-based models is 97% (Random Forest), while smell-based approach has a score of 83% (Random Forest). Analysing MCC, the difference is higher. This measure takes into account true and false positives, and negatives, the best result obtained by the smell-based approach is 0.66 and the best obtained by the vocabulary-based approach is 0.94.

However, the results obtained in the cross-project validation show that the test smell-based approach reaches better results. In the intra-project context, the test smell-based approach obtained 74% of recall (LR), and the vocabulary-based one achieved at most 57% (KNN). Considering the best classifiers in the inter-project context, the performance is similar, the smell-based approach obtained a recall of at most 55% (SVM), against 56% of the vocabulary-based (LDA). But if we consider all the classifiers, we observe that smell-based approach has a general better performance in the cross-project validation than the vocabulary-based approach.

Answer to RQ₃: The performance of the vocabulary-based models are better than the performance of the test smell-based ones using the training and testing dataset. But in the cross-project validation scenario, the smell-based approach obtains significant better results in the intra-project and inter-project contexts.

Implications: The results obtained in our study corroborate the ones presented in our previous work [4]. The vocabulary-based approach do not have a good performance in cross-project validation. Our results shows that the use of test smells can be a good alternative for overcoming some limitations of the vocabulary-based approach, and smell-based models can be more generalizable.

7 THREATS TO VALIDITY

Threats to construct validity are related to the metrics used to evaluate the results. But to minimize this threat we adopted the most common measures used in the ML field to evaluate the classifiers.

During the test code pre-processing to obtain the test smells in some cases `tsDetect` [24] could not identify the production class, then the smells could not be extracted, which can compromise the result.

Threats to internal validity may comprise the results when relating independent and dependent variables. The absence of non-flaky class on the cross-project dataset made impossible to obtain precision and other metrics to use as a benchmark. This should be evaluated in future work.

External validity is connected to the generalization of the obtained results. As in similar studies, we cannot generalize the results, once this study targeted the Java language and a limited set of project domains. Considering the cross-project validation, the results obtained in the intra-project context show an expressive

Table 5: Information gain of each feature of the model.

Pos.	Features	Inf. Gain	Total	Flaky tests	% Flaky tests	Non-Flaky tests	% Non-Flaky tests
1	LOC	0.2545	2777	1377	49.59%	1400	50.41%
2	Assertion Roulette	0.0832	1389	968	69.69%	421	30.31%
3	Smells Count	0.0271	2655	1356	51.07%	1299	48.93%
4	Sleepy Test	0.0195	112	105	93.75%	7	6.25%
5	General Fixture	0.0160	267	61	22.85%	206	77.15%
6	Duplicate Assert	0.0155	376	269	71.54%	107	28.46%
7	Constructor Initialization	0.0109	68	63	92.65%	5	7.35%
8	Redundant Print	0.0106	58	55	94.83%	3	5.17%
9	Sensitive Equality	0.0059	129	95	73.64%	34	26.36%
10	Lazy Test	0.0055	1788	817	45.69%	971	54.31%
11	Resource Optimism	0.0043	75	17	22.67%	58	77.33%
12	Conditional Test Logic	0.0042	356	219	61.52%	137	38.48%
13	Unknown Test	0.0021	544	234	43.01%	310	56.99%
14	Verbose Test	0.0018	7	7	100%	0	0.00%
15	Magic Number Test	0.0011	411	227	55.23%	184	44.77%
16	Mystery Guest	0.0006	124	71	57.26%	53	42.74%
17	Eager Test	0.0003	970	496	51.13%	474	48.87%
18	Redundant Assertion	0.0000	8	4	50.00%	4	50.00%
19	Default Test	0.0000	0	0	0.00%	0	0.00%
20	Empty Test	0.0000	0	0	0.00%	0	0.00%
21	Ignored Test	0.0000	0	0	0.00%	0	0.00%

Table 6: Smell count distribution.

Smells Count	Non-Flaky	% Non-Flaky	Flaky	% Flaky
0	101	83%	21	17%
1	469	61%	305	39%
2	327	50%	329	50%
3	264	46%	308	4%
4	159	35%	301	65%
5	63	46%	73	54%
6	12	29%	30	71%
7	4	33%	8	67%
8	1	33%	2	67%

Table 7: Vocabulary-based classifiers' performance.

Algorithm	Precision	Recall	F ₁	MCC	AUC
Random Forest	0.97	0.97	0.97	0.93	0.99
Perceptron	0.97	0.97	0.97	0.93	0.99
Decision Tree	0.94	0.94	0.94	0.87	0.94
Naive Bayes	0.95	0.95	0.95	0.90	0.95
SVM	0.97	0.97	0.97	0.94	0.99
LR	0.97	0.97	0.97	0.94	0.99
LDA	0.87	0.87	0.87	0.74	0.88
KNN	0.93	0.93	0.93	0.86	0.93

Table 8: Cross-project vocabulary-based performance.

Algorithm	Intra-Project			Inter-Project		
	Recall	TP	FN	Recall	TP	FN
Decision Tree	0.31	11	24	0.39	47	73
LDA	0.29	10	25	0.56	67	53
LR	0.20	7	28	0.30	36	84
Random Forest	0.17	6	29	0.29	35	85
Naive Bayes	0.17	6	29	0.13	15	105
SVM	0.09	3	32	0.17	20	100
KNN	0.57	20	15	0.23	27	93
Perceptron	0.34	12	23	0.33	40	80

reduction in the performance if compared with the results reported in [4]. This could be a result of the cross-project validation dataset, which is smaller and should be increased in future works to better understand the performance of prediction.

8 CONCLUSION

Regression tests are an important practice for supporting continuous integration and delivery, and flaky tests are disturbing to this process. Consequently, the proper identification and prevention of test flakiness are important topics pursued by researchers and practitioners.

This paper investigated the use of test smells as features to predict flaky tests. To this end, we conducted an empirical study to evaluate the performance of the test smell-based approach for prediction of flakiness. Such a performance was measured considering

different evaluation indicators, and cross-project validation. We also identified the test smells most strongly associated with test flakiness prediction, and compared the test smell-based approach with the state-of-the-art vocabulary-based approach.

As result, we observed that test smells are a potentially good predictors of test flakiness; the results of intra- and inter-project are promising. Some test smells like *Sleepy Test* and *Constructor Initialization* are strongly associated with flakiness. Compared to the vocabulary-based approach, the test smell-based one obtained models that have in the best case, a precision 14% lower. In the cross-project validation, the test smell-based approach performs better in the intra- and inter- project contexts.

This study opens the possibility to use smell-based models for the prediction of test flakiness. As a future work it is possible to expand the training dataset by adding samples of other projects of other contexts and programming languages. We also intend to explore other static and dynamic features in combination with test smells.

ACKNOWLEDGMENTS

This work is partially supported by CNPq (Andre T. Endo grant nr. 420363/2018-1 and Silvia Regina Vergilio grant nr. 305968/2018-1) and by IN2 Institute (Bruno Henrique Pachulski Camara grant nr. 002/2021).

REFERENCES

- [1] Azeem Ahmad, Ola Leifler, and K. Sandahl. 2020. An Evaluation of Machine Learning Methods for Predicting Flaky Tests. In *Proceedings of the 8th International Workshop on Quantitative Approaches to Software Quality (QuASoQ 2020)*.
- [2] Abdulrahman Alshammari, Christopher Morris, Michael Hilton, and Jonathan Bell. 2021. FlakeFlagger: Predicting Flakiness Without Rerunning Tests. In *43rd IEEE/ACM International Conference on Software Engineering, ICSE 2021, Madrid, Spain, 22-30 May 2021*. IEEE, 1572–1584. <https://doi.org/10.1109/ICSE43902.2021.00140>
- [3] Jonathan Bell, Owolabi Legunsen, Michael Hilton, Lamyaa Eloussi, Tiffany Yung, and Darko Marinov. 2018. DeFlaker: Automatically Detecting Flaky Tests. In *40th International Conference on Software Engineering (40 ed.)* (Gothenburg, Sweden). ACM, New York, NY, USA, 433–444. <https://doi.org/10.1145/3180155.3180164>
- [4] Bruno Henrique Pachulski Camara, Marco Aurélio Graciotto Silva, André Takeshi Endo, and Silvia Regina Vergilio. 2021. What is the Vocabulary of Flaky Tests? An Extended Replication. In *29th IEEE/ACM International Conference on Program Comprehension, ICPC 2021, Madrid, Spain, May 20-21, 2021*. IEEE, 444–454. <https://doi.org/10.1109/ICPC52881.2021.00052>
- [5] Arie Deursen, Leon M.F. Moonen, A. Bergh, and Gerard Kok. 2001. *Refactoring Test Code*. Technical Report. NLD.
- [6] Moritz Eck, Fabio Palomba, Marco Castelluccio, and Alberto Bacchelli. 2019. Understanding flaky tests: the developer’s perspective. In *27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (27 ed.)* (Tallinn, Estonia). ACM, New York, NY, USA, 830–840. <https://doi.org/10.1145/3338906.3338945>
- [7] Martin Fowler. 2011. Eradicating Non-Determinism in Tests. Web page. <https://bit.ly/2ZlJ63W> Accessed: 2021-07-23.
- [8] Martin Fowler. 2020. *Refatoração – 2ª edição: Aperfeiçoando o design de códigos existentes*. Novatec Editora. <https://books.google.com.br/books?id=cZTeDwAAQBAJ>
- [9] William Goddard. 2018. What’s the Best Programming Language for Machine Learning Applications? Web page. <https://cutt.ly/Hk1flto> Accessed: 2021-02-16.
- [10] Guillaume Haben, Sarra Habchi, Mike Papadakis, Maxime Cordy, and Yves Le Traon. 2021. A Replication Study on the Usability of Code Vocabulary in Predicting Flaky Tests. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. 219–229. <https://doi.org/10.1109/MSR52588.2021.00034>
- [11] Kim Herzig and Nachiappan Nagappan. 2015. Empirically Detecting False Test Alarms Using Association Rules. In *37th International Conference on Software Engineering (37 ed.)* (Florence, Italy). IEEE, Piscataway, NJ, USA, 39–48. <https://doi.org/10.5555/2819009.2819018>
- [12] Tariq M. King, Dionny Santiago, Justin Phillips, and Peter J. Clarke. 2018. Towards a Bayesian Network Model for Predicting Flaky Automated Tests. In *2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)* (Lisbon, Portugal). IEEE, 100–107. <https://doi.org/10.1109/QRS-C.2018.00031>
- [13] Wing Lam, Patrice Godefroid, Suman Nath, Anirudh Santhiar, and Suresh Thummalapeda. 2019. Root Causing Flaky Tests in a Large-Scale Industrial Setting. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Beijing, China). ACM, New York, NY, USA, 101–111. <https://doi.org/10.1145/3293882.3330570>
- [14] Wing Lam, Kivanç Muslu, Hitesh Sajani, and Suresh Thummalapeda. 2020. A study on the lifecycle of flaky tests. In *ICSE ’20: 42nd International Conference on Software Engineering, Seoul, South Korea, 27 June - 19 July, 2020*, Gregg Rothermel and Doo-Hwan Bae (Eds.). ACM, 1471–1482. <https://doi.org/10.1145/3377811.3381749>
- [15] Wing Lam, Reed Oei, August Shi, Darko Marinov, and Tao Xie. 2019. iDFlakes: A Framework for Detecting and Partially Classifying Flaky Tests. In *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)* (12 ed.) (Xi’an, China). IEEE, 312–322. <https://doi.org/10.1109/ICST.2019.00038>
- [16] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An Empirical Analysis of Flaky Tests. In *22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (22 ed.)* (Hong Kong, China). ACM, New York, NY, USA, 643–653. <https://doi.org/10.1145/2635868.2635920>
- [17] Atif M. Memon, Zebao Gao, Bao N. Nguyen, Sanjeev Dhandia, Eric Nickell, Rob Siemborski, and John Micco. 2017. Taming Google-Scale Continuous Testing. In *39th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice Track, ICSE-SEIP 2017, Buenos Aires, Argentina, May 20-28, 2017*. IEEE Computer Society, 233–242. <https://doi.org/10.1109/ICSE-SEIP.2017.16>
- [18] Gerard Meszaros. 2007. *xUnit Test Patterns: Refactoring Test Code*. Pearson Education. <https://books.google.com.br/books?id=-izOiCELABQC>
- [19] John Micco. 2016. Flaky Tests at Google and How We Mitigate Them. Web page. <https://bit.ly/2Nz4fF5>
- [20] Jesus Moran, Cristian Augusto Alonso, Antonia Bertolino, Claudio de la Riva, and Javier Tuya. 2020. FlakyLoc: Flakiness Localization for Reliable Test Suites in Web Applications. *Journal of Web Engineering* (06 2020). <https://doi.org/10.13052/jwe1540-9589.1927>
- [21] Jason Palmer. 2019. Test Flakiness – Methods for identifying and dealing with flaky tests. Web page. <https://bit.ly/2NbesYv> Accessed: 2021-02-16.
- [22] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12, 85 (2011), 2825–2830.
- [23] Anthony Peruma, Khalid Almalki, Christian D. Newman, Mohamed Wiem Mkaouer, Ali Ouni, and Fabio Palomba. 2019. On the Distribution of Test Smells in Open Source Android Applications: An Exploratory Study. In *Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering (Toronto, Ontario, Canada) (CASCON ’19)*. IBM Corp., USA, 193–202.
- [24] Anthony Peruma, Khalid Almalki, Christian D. Newman, Mohamed Wiem Mkaouer, Ali Ouni, and Fabio Palomba. 2020. TSdetect: An Open Source Test Smells Detection Tool. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Virtual Event, USA) (ESEC/FSE 2020)*. Association for Computing Machinery, New York, NY, USA, 1650–1654. <https://doi.org/10.1145/3368089.3417921>
- [25] Gustavo Pinto, Breno Miranda, Supun Dissanayake, Marcelo d’Amorim, Christoph Treude, and Antonia Bertolino. 2020. What is the Vocabulary of Flaky Tests?. In *17th International Conference on Mining Software Repositories*. IEEE / ACM, Seoul, South Korea. <https://doi.org/10.1145/3379597.3387482>
- [26] August Shi, Wing Lam, Reed Oei, Tao Xie, and Darko Marinov. 2019. iFixFlakes: A Framework for Automatically Fixing Order-Dependent Flaky Tests. In *27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (27 ed.)* (Tallinn, Estonia). ACM, New York, NY, USA, 545–555. <https://doi.org/10.1145/3338906.3338925>
- [27] Davide Spadini, Martin Schvarcbacher, Ana-Maria Oprescu, Magiel Bruntink, and Alberto Bacchelli. 2020. Investigating Severity Thresholds for Test Smells. In *Proceedings of the 17th International Conference on Mining Software Repositories (Seoul, Republic of Korea) (MSR ’20)*. Association for Computing Machinery, New York, NY, USA, 311–321. <https://doi.org/10.1145/3379597.3387453>
- [28] Zolfaghari, Behrouz, Parizi Reza M., Gautam Srivastava, and Yoseph Hailemariam. 2020. Root causing, detecting, and fixing flaky tests: State of the art and future roadmap. *Software: Practice and Experience* (2020). <https://doi.org/10.1002/spe.2929> arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.2929